



TRƯỜNG ĐẠI HỌC QUY NHƠN

BÀI TIỂU LUẬN CUỐI KỲ HỌC PHẦN PHÂN TÍCH THỐNG KÊ VỚI R

BOOTSTRAP VÀ K-FOLD CROSS-VALIDATION TRONG BÀI TOÁN DỰ ĐOÁN BỆNH GAN

Trình độ đào tạo: Thạc sĩ
Học viên thực hiện: Nguyễn Hồ Bảo Thiên
Lớp: Khoa học dữ liệu K28B
Giảng viên hướng dẫn: TS. Nguyễn Đặng Thiên Thu

Gia Lai, 2026

Mục lục

1	Đặt vấn đề	2
2	Cơ sở lý thuyết	3
2.1	Phương pháp Bootstrap	3
2.2	Phương pháp K-fold Cross-Validation	5
3	Dữ liệu và tiền xử lý	8
4	Phân tích khám phá dữ liệu	10
4.1	Phân bố biến mục tiêu	10
4.2	Tương quan giữa các biến	10
4.3	Phân bố các biến sinh hoá và biến đổi log	11
4.4	Đa cộng tuyến và hệ số VIF	13
4.5	Thống kê mô tả theo nhóm	15
4.6	Lựa chọn đặc trưng	17
5	Thiết kế và lựa chọn mô hình	18
6	Lựa chọn ngưỡng phân loại	20
7	Đánh giá trên tập kiểm tra	21
8	Khoảng tin cậy Bootstrap	23
9	Kết luận	24
	Tài liệu tham khảo	26

1 Đặt vấn đề

Các bệnh lý về gan thường tiến triển âm thầm trong thời gian dài và chỉ biểu hiện triệu chứng rõ rệt khi bệnh đã bước sang giai đoạn nặng, làm giảm đáng kể hiệu quả của các biện pháp điều trị. Vì vậy, việc phát hiện sớm nguy cơ mắc bệnh thông qua các xét nghiệm sinh hóa thường quy, chẳng hạn như bilirubin, men gan và albumin, có ý nghĩa quan trọng trong hỗ trợ chẩn đoán và can thiệp kịp thời.

Bộ dữ liệu *Indian Liver Patient Dataset* (ILPD) gồm 583 mẫu dữ liệu, mỗi mẫu tương ứng với một bệnh nhân và bao gồm các chỉ số sinh hóa cùng nhãn chẩn đoán xác định. Từ đó, bài toán được đặt ra là xây dựng một mô hình phân loại nhị phân nhằm dự đoán tình trạng mắc bệnh gan dựa trên kết quả xét nghiệm. Đây là một bài toán điển hình trong học máy ứng dụng vào lĩnh vực y tế, nơi không chỉ yêu cầu mô hình đạt hiệu năng cao mà còn phải có khả năng tổng quát hóa tốt trên dữ liệu chưa từng được quan sát.

Trong thực tế, việc chỉ huấn luyện một mô hình và báo cáo một giá trị hiệu năng trên một tập dữ liệu là chưa đủ để khẳng định chất lượng của mô hình. Các chỉ số như Accuracy, F1-score hay ROC-AUC chỉ là những ước lượng thu được từ một mẫu dữ liệu hữu hạn và có thể thay đổi khi áp dụng trên những tập dữ liệu khác. Do đó, quá trình đánh giá cần giải quyết hai vấn đề quan trọng sau:

1. Trong số các mô hình và cấu hình siêu tham số được thử nghiệm, mô hình nào có khả năng tổng quát hóa tốt nhất? Để trả lời câu hỏi này, cần sử dụng một quy trình đánh giá khách quan nhằm ước lượng hiệu năng của mô hình trên dữ liệu chưa từng được sử dụng để huấn luyện ở mỗi lần đánh giá. *K-fold Cross-Validation* là phương pháp được sử dụng để đánh giá và hỗ trợ lựa chọn mô hình phù hợp.
2. Sau khi lựa chọn được mô hình, mức độ tin cậy của các chỉ số đánh giá là bao nhiêu? Thay vì chỉ báo cáo một giá trị ước lượng điểm (*point estimate*), cần ước lượng thêm khoảng tin cậy (*confidence interval*) để phản ánh mức độ biến thiên của các chỉ số đánh giá. Phương pháp *Bootstrap* được sử dụng nhằm ước lượng các khoảng tin cậy này.

Như vậy, *K-fold Cross-Validation* và *Bootstrap* đảm nhận hai vai trò bổ sung cho nhau trong quá trình đánh giá mô hình. Trong khi *K-fold Cross-Validation* được sử dụng để ước lượng khả năng tổng quát hóa và hỗ trợ lựa chọn mô hình, thì *Bootstrap* cho phép ước lượng khoảng tin cậy của các chỉ số đánh giá, từ đó phản ánh mức độ bất định của hiệu năng mô hình. Báo cáo này trình bày cơ sở lý thuyết của hai phương pháp, đồng thời xây dựng một quy trình học máy hoàn chỉnh trên bộ dữ liệu ILPD, bao gồm tiền xử lý dữ liệu, lựa chọn đặc trưng, tính

chỉnh siêu tham số, tối ưu ngưỡng phân loại và ước lượng khoảng tin cậy cho hiệu năng của mô hình.

2 Cơ sở lý thuyết

Bootstrap và K-fold Cross-Validation đều thuộc nhóm các kỹ thuật *resampling* (lấy mẫu lại), được phát triển nhằm giải quyết một vấn đề cơ bản trong thống kê suy luận. Trong thực tế, nhà nghiên cứu chỉ có thể quan sát một mẫu hữu hạn được rút ra từ tổng thể, trong khi các đại lượng thống kê được tính từ mẫu đó, chẳng hạn như giá trị trung bình, hệ số hồi quy hay độ chính xác của mô hình, đều là những biến ngẫu nhiên. Nếu lấy một mẫu khác từ cùng tổng thể, các đại lượng này nhiều khả năng sẽ nhận những giá trị khác.

Do đó, câu hỏi quan trọng không chỉ là giá trị ước lượng bằng bao nhiêu, mà còn là mức độ biến thiên của ước lượng đó khi dữ liệu thay đổi. Trong nhiều trường hợp, phân phối lấy mẫu của đại lượng quan tâm không thể xác định chính xác hoặc quá phức tạp để suy ra bằng lý thuyết. Khi đó, các phương pháp *resampling* cho phép xấp xỉ phân phối này bằng cách tạo nhiều mẫu lấy lại từ dữ liệu ban đầu và quan sát sự thay đổi của đại lượng quan tâm trên các mẫu đó.

2.1 Phương pháp Bootstrap

Bootstrap là một phương pháp *resampling* do Bradley Efron đề xuất vào năm 1979, được sử dụng để ước lượng độ chính xác của các đại lượng thống kê, chẳng hạn như sai số chuẩn và khoảng tin cậy, mà không cần giả định trước về phân phối của tổng thể cũng như không yêu cầu thu thập thêm dữ liệu.

Nền tảng lý thuyết. Xét tham số tổng thể $\theta = \theta(F)$, trong đó F là hàm phân phối chưa biết của tổng thể. Từ mẫu quan sát $x_1, \dots, x_n$, ta thu được ước lượng $\hat{\theta} = \theta(\hat{F})$. Để đánh giá độ chính xác của $\hat{\theta}$, cần biết *phân phối lấy mẫu* của nó, tức phân phối của $\hat{\theta}$ nếu quá trình lấy mẫu kích thước n từ F được lặp lại vô số lần. Tuy nhiên, điều này không khả thi trong thực tế vì hàm phân phối F không được biết trước và thông thường chỉ có một mẫu quan sát duy nhất.

Ý tưởng cốt lõi của Bootstrap là thay thế tổng thể chưa biết F bằng *phân phối thực nghiệm* $\hat{F}$, tức phân phối rời rạc gán xác suất $1/n$ cho mỗi quan sát trong mẫu. Theo định lý Glivenko–Cantelli, $\hat{F}$ hội tụ đều về F khi kích thước mẫu tăng, do đó $\hat{F}$ có thể được xem là một xấp xỉ hợp lý của F . Khi đó, việc rút một mẫu kích thước n từ $\hat{F}$ tương đương với việc lấy mẫu ngẫu nhiên **có hoàn lại** từ dữ liệu gốc. Đây chính là nội dung của *nguyên lý plug-in*: sử dụng phân phối thực nghiệm để xấp xỉ phân phối của tổng thể, từ đó xấp xỉ phân phối lấy

mẫu của θ bằng phân phối của các ước lượng bootstrap $\hat{\theta}^*$ thu được từ các mẫu lấy lại.

Thuật toán.

1. Từ mẫu gốc có kích thước n , lấy mẫu ngẫu nhiên **có hoàn lại** cũng có kích thước n , thu được một mẫu bootstrap.
2. Tính đại lượng thống kê quan tâm $\hat{\theta}_b^*$ trên mẫu bootstrap đó.
3. Lặp lại hai bước trên B lần, với B thường từ khoảng 1 000 đến vài nghìn, thu được tập các ước lượng bootstrap $\hat{\theta}_1^*, \dots, \hat{\theta}_B^*$.
4. Sử dụng phân phối thực nghiệm của các ước lượng bootstrap để ước lượng sai số chuẩn và khoảng tin cậy của θ .

Sai số chuẩn và khoảng tin cậy. Sai số chuẩn bootstrap được ước lượng bằng độ lệch chuẩn của B giá trị thống kê bootstrap:

$$\widehat{SE}_{boot} = \sqrt{\frac{1}{B-1} \sum_{b=1}^B \left(\hat{\theta}_b^* - \bar{\theta}^* \right)^2}, \quad \bar{\theta}^* = \frac{1}{B} \sum_{b=1}^B \hat{\theta}_b^*.$$

Có nhiều phương pháp để xây dựng khoảng tin cậy từ các mẫu bootstrap, trong đó *phương pháp phân vị (Percentile Bootstrap)* là cách đơn giản và được sử dụng phổ biến. Với mức tin cậy $1 - \alpha$, hai đầu mút của khoảng tin cậy được xác định lần lượt bởi các phân vị mức $\alpha/2$ và $1 - \alpha/2$ của tập B giá trị bootstrap. Chẳng hạn, với mức tin cậy 95%, khoảng tin cậy được xác định bởi phân vị 2,5% và phân vị 97,5%.

Ưu điểm.

1. *Không yêu cầu giả định trước về phân phối của tổng thể.* Nhiều phương pháp xây dựng khoảng tin cậy cổ điển dựa trên giả định dữ liệu tuân theo phân phối chuẩn hoặc dựa vào các kết quả tiệm cận khi kích thước mẫu đủ lớn. Bootstrap không yêu cầu các giả định này, do đó đặc biệt hữu ích đối với dữ liệu có phân phối lệch hoặc trong những trường hợp khó áp dụng các công thức lý thuyết.
2. *Áp dụng được cho nhiều đại lượng thống kê.* Nhiều chỉ số đánh giá trong học máy, chẳng hạn như Recall, Precision hay Macro F1-score, không có công thức giải tích đơn giản để tính sai số chuẩn hoặc khoảng tin cậy. Bootstrap chỉ cần tính lại đại lượng quan tâm trên từng mẫu bootstrap, nhờ đó có thể áp dụng thống nhất cho hầu hết các thống kê.

3. *Không cần thu thập thêm dữ liệu.* Toàn bộ thông tin về mức độ biến thiên của ước lượng được khai thác từ chính mẫu quan sát ban đầu. Đây là một ưu điểm quan trọng trong các nghiên cứu y sinh, nơi việc mở rộng cỡ mẫu thường tốn kém cả về chi phí lẫn thời gian.
4. *Cho phép xấp xỉ phân phối lấy mẫu của thống kê.* Kết quả của Bootstrap không chỉ là sai số chuẩn hoặc khoảng tin cậy mà còn bao gồm toàn bộ tập các giá trị bootstrap. Nhờ đó, có thể khảo sát hình dạng của phân phối lấy mẫu xấp xỉ, đánh giá mức độ đối xứng, độ phân tán và tính ổn định của ước lượng.

Hạn chế.

1. *Chi phí tính toán cao.* Quá trình tính toán đại lượng thống kê phải được lặp lại hàng nghìn lần trên các mẫu bootstrap. Chi phí này càng tăng nếu mỗi lần lặp đều phải huấn luyện lại mô hình thay vì chỉ tính lại các chỉ số đánh giá từ những dự đoán đã có.
2. *Hiệu quả phụ thuộc vào kích thước và chất lượng mẫu.* Phân phối thực nghiệm $\hat{F}$ chỉ xấp xỉ tốt phân phối tổng thể F khi mẫu quan sát đủ lớn và có tính đại diện. Với cỡ mẫu nhỏ, các mẫu bootstrap chỉ được tạo từ một số lượng quan sát hạn chế, khiến phân phối bootstrap có thể chưa phản ánh chính xác phân phối lấy mẫu thực sự. Do đó, sai số chuẩn và khoảng tin cậy ước lượng có thể kém chính xác.
3. *Giả định các quan sát độc lập và cùng phân phối.* Bootstrap cơ bản giả định dữ liệu được lấy độc lập và cùng phân phối (i.i.d.). Vì vậy, phương pháp này không thể áp dụng trực tiếp cho các dữ liệu có cấu trúc phụ thuộc, chẳng hạn như chuỗi thời gian hoặc dữ liệu phân cụm theo bệnh viện hay khu vực địa lý.
4. *Không tạo ra thông tin mới.* Bootstrap chỉ khai thác lại thông tin từ mẫu quan sát ban đầu. Nếu bản thân mẫu gốc không đại diện cho tổng thể, chẳng hạn do sai lệch chọn mẫu, thì các khoảng tin cậy và ước lượng thu được vẫn có thể bị sai lệch so với giá trị thực của tổng thể.

2.2 Phương pháp K-fold Cross-Validation

K-fold Cross-Validation, hay kiểm định chéo k phần, là một kỹ thuật *resampling* **không hoàn lại**, được sử dụng để ước lượng khả năng tổng quát hóa của mô hình trên dữ liệu chưa từng được sử dụng trong quá trình huấn luyện. Nhờ đó, phương pháp giúp đánh giá khách quan hơn hiệu năng của mô hình so với việc đánh giá trực tiếp trên tập dữ liệu huấn luyện.

Nền tảng lý thuyết. Mục tiêu của một mô hình học máy là dự đoán tốt trên dữ liệu mới, thay vì chỉ khớp với dữ liệu đã quan sát. Nếu đánh giá mô hình trên chính tập dữ liệu dùng để huấn luyện, kết quả thu được thường quá lạc quan vì mô hình có thể đã học cả những dao động ngẫu nhiên đặc trưng của mẫu huấn luyện, hay còn gọi là hiện tượng *overfitting*. Một cách khắc phục đơn giản là tách riêng một tập kiểm tra độc lập. Tuy nhiên, phương pháp này làm giảm lượng dữ liệu dành cho huấn luyện và kết quả đánh giá phụ thuộc đáng kể vào một lần chia dữ liệu ngẫu nhiên, đặc biệt khi kích thước mẫu nhỏ. K-fold Cross-Validation khắc phục hạn chế này bằng cách luân phiên sử dụng các phần dữ liệu làm tập huấn luyện và tập kiểm tra, sao cho mỗi quan sát được sử dụng đúng một lần để kiểm tra và $k - 1$ lần để huấn luyện.

Thuật toán.

1. Chia ngẫu nhiên tập dữ liệu thành k phần rời nhau có kích thước xấp xỉ bằng nhau, gọi là các *fold*.
2. Với mỗi $i = 1, \dots, k$, sử dụng $k - 1$ fold để huấn luyện mô hình và fold còn lại làm tập kiểm tra.
3. Tính sai số dự đoán trên từng fold, sau đó lấy trung bình:

$$CV_{(k)} = \frac{1}{k} \sum_{i=1}^k Err_i,$$

trong đó Err_i là giá trị trung bình của hàm mất mát trên fold thứ i . Nếu ký hiệu $\mathcal{F}_i$ là tập các quan sát thuộc fold thứ i , ta có

$$Err_i = \frac{1}{|\mathcal{F}_i|} \sum_{j \in \mathcal{F}_i} L(y_j, \hat{f}^{(-i)}(x_j)),$$

với L là hàm mất mát và $\hat{f}^{(-i)}$ là mô hình được huấn luyện khi loại bỏ fold thứ i .

Ví dụ minh họa. Xét bài toán phân loại bệnh gan trên bộ dữ liệu ILPD gồm $n = 583$ quan sát bằng mô hình hồi quy logistic. Với $k = 5$, dữ liệu được chia thành 5 fold, mỗi fold gồm khoảng 117 quan sát. Ở mỗi lần lặp, mô hình được huấn luyện trên khoảng 466 quan sát thuộc 4 fold và được đánh giá trên fold còn lại. Quá trình được lặp lại 5 lần để mỗi fold lần lượt đóng vai trò tập kiểm tra. Giá trị trung bình của năm lần đánh giá được sử dụng để ước lượng hiệu năng dự đoán của mô hình, trong khi sự khác biệt giữa các fold phản ánh mức độ ổn định của mô hình đối với các cách chia dữ liệu khác nhau.

Biến thể Stratified K-fold. Khi biến mục tiêu bị mất cân bằng lớp, việc chia dữ liệu hoàn toàn ngẫu nhiên có thể tạo ra những fold có tỷ lệ lớp khác biệt

đáng kể so với toàn bộ dữ liệu, thậm chí không chứa lớp thiểu số. Điều này làm cho kết quả đánh giá có phương sai lớn và thiếu ổn định. **Stratified K-fold** khắc phục vấn đề này bằng cách duy trì tỷ lệ các lớp trong mỗi fold gần giống với tỷ lệ của toàn bộ tập dữ liệu. Đối với ILPD, tỷ lệ giữa nhóm mắc bệnh và không mắc bệnh xấp xỉ 71% và 29%, vì vậy mỗi fold đều được tạo sao cho phản ánh gần đúng tỷ lệ này. Nhờ đó, kết quả đánh giá giữa các fold ổn định và có tính đại diện cao hơn. Đây cũng là biến thể được sử dụng trong toàn bộ phần thực nghiệm của báo cáo.

Ưu điểm.

1. *Ước lượng khách quan hơn về khả năng tổng quát hóa.* Mỗi quan sát đều được dự đoán bởi một mô hình chưa từng sử dụng chính quan sát đó trong quá trình huấn luyện, nhờ đó giảm sự lạc quan của các chỉ số đánh giá.
2. *Sử dụng dữ liệu hiệu quả.* Toàn bộ các quan sát đều lần lượt tham gia vào cả quá trình huấn luyện và kiểm tra, giúp tận dụng tối đa dữ liệu, đặc biệt khi kích thước mẫu hạn chế.
3. *Đánh giá tính ổn định của mô hình.* Ngoài giá trị trung bình của các chỉ số đánh giá, K-fold Cross-Validation còn cung cấp kết quả trên từng fold, qua đó cho phép quan sát mức độ dao động của mô hình dưới các cách chia dữ liệu khác nhau.
4. *Áp dụng cho nhiều loại mô hình.* Phương pháp chỉ yêu cầu mô hình có khả năng huấn luyện và dự đoán, do đó có thể áp dụng thống nhất cho hầu hết các thuật toán học máy.

Hạn chế.

1. *Chi phí tính toán cao.* Mô hình phải được huấn luyện k lần thay vì một lần. Chi phí này còn tăng đáng kể khi K-fold Cross-Validation được sử dụng bên trong quá trình dò siêu tham số (Grid Search hoặc Random Search).
2. *Đánh đổi giữa bias và variance.* Khi k nhỏ, mỗi mô hình được huấn luyện trên ít dữ liệu hơn nên ước lượng sai số có xu hướng thiên lệch (bias) lớn hơn. Ngược lại, khi k tăng, các tập huấn luyện trở nên rất giống nhau, khiến phương sai của ước lượng Cross-Validation có xu hướng tăng.
3. *Các lần đánh giá không độc lập.* Các tập huấn luyện ở các fold khác nhau chia sẻ phần lớn các quan sát. Vì vậy, độ lệch chuẩn giữa các kết quả trên từng fold không phải là sai số chuẩn của ước lượng Cross-Validation mà chỉ nên được xem như một chỉ báo về mức độ ổn định của mô hình.

4. Có thể dẫn đến *selection bias* trong lựa chọn mô hình. Nếu cùng một kết quả K-fold Cross-Validation được sử dụng để so sánh rất nhiều mô hình hoặc cấu hình siêu tham số, mô hình được lựa chọn có thể vô tình phù hợp với cách chia fold cụ thể hơn là thật sự có khả năng tổng quát hóa tốt. Trong trường hợp này, nên sử dụng một tập kiểm tra độc lập hoặc Nested Cross-Validation để đánh giá cuối cùng.

3 Dữ liệu và tiền xử lý

Nguồn dữ liệu và cấu trúc

Bộ dữ liệu ILPD được thu thập tại vùng đông bắc bang Andhra Pradesh, Ấn Độ, gồm 583 hồ sơ bệnh nhân với 10 biến giải thích là tuổi, giới tính và tám chỉ số sinh hoá. Nhãn gốc mã hoá giá trị 1 cho trường hợp có bệnh và giá trị 2 cho trường hợp không bệnh; nhãn này được đổi về quy ước 1 và 0 để thuận tiện cho các công thức đếm ở phần sau.

```
col_names <- c(
  "Age", "Gender", "Total_Bilirubin", "Direct_Bilirubin",
  "Alkaline_Phosphotase", "Alamine_Aminotransferase",
  "Aspartate_Aminotransferase", "Total_Protiens", "Albumin",
  "Albumin_and_Globulin_Ratio", "target"
)

df <- read.csv("Indian Liver Patient Dataset (ILPD).csv",
  header = FALSE, col.names = col_names,
  stringsAsFactors = FALSE)

# Nhãn gốc: 1 = có bệnh, 2 = không bệnh. Đổi về 1 / 0.
df$target <- ifelse(df$target == 2, 0, 1)

cat("Kích thước dữ liệu:", nrow(df), "dòng x", ncol(df), "cột\n")

## Kích thước dữ liệu: 583 dòng x 11 cột

cat("Phân bố target:", sum(df$target == 1), "có bệnh /",
  sum(df$target == 0), "không bệnh\n")

## Phân bố target: 416 có bệnh / 167 không bệnh
```

```
str(df)

## 'data.frame': 583 obs. of 11 variables:
## $ Age : int 65 62 62 58 72 46 26 29 17 55 ...
## $ Gender : chr "Female" "Male" "Male" "Male" ...
## $ Total_Bilirubin : num 0.7 10.9 7.3 1 3.9 1.8 0.9 0.9 0.9 0.7 ...
## $ Direct_Bilirubin : num 0.1 5.5 4.1 0.4 2 0.7 0.2 0.3 0.3 0.2 ...
## $ Alkaline_Phosphotase : int 187 699 490 182 195 208 154 202 202 290 ...
## $ Alamine_Aminotransferase : int 16 64 60 14 27 19 16 14 22 53 ...
## $ Aspartate_Aminotransferase : int 18 100 68 20 59 14 12 11 19 58 ...
## $ Total_Protiens : num 6.8 7.5 7 6.8 7.3 7.6 7 6.7 7.4 6.8 ...
## $ Albumin : num 3.3 3.2 3.3 3.4 2.4 4.4 3.5 3.6 4.1 3.4 ...
## $ Albumin_and_Globulin_Ratio : num 0.9 0.74 0.89 1 0.4 1.3 1 1.1 1.2 1 ...
## $ target : num 1 1 1 1 1 1 1 1 0 1 ...
```

Thống kê mô tả và giá trị khuyết

```
summary(df)

##      Age      Gender  Total_Bilirubin  Direct_Bilirubin
## Min.   : 4.00    Length :583    Min.   : 0.400    Min.   : 0.100
## 1st Qu.:33.00    N.unique : 2    1st Qu.: 0.800    1st Qu.: 0.200
## Median :45.00    N.blank  : 0    Median : 1.000    Median : 0.300
## Mean   :44.75    Min.nchar: 4    Mean   : 3.299    Mean   : 1.486
## 3rd Qu.:58.00    Max.nchar: 6    3rd Qu.: 2.600    3rd Qu.: 1.300
## Max.   :90.00                Max.   :75.000    Max.   :19.700
##
## Alkaline_Phosphotase Alamine_Aminotransferase
## Min.   : 63.0      Min.   : 10.00
## 1st Qu.:175.5      1st Qu.: 23.00
## Median :208.0      Median : 35.00
## Mean   :290.6      Mean   : 80.71
## 3rd Qu.:298.0      3rd Qu.: 60.50
## Max.   :2110.0     Max.   :2000.00
##
## Aspartate_Aminotransferase Total_Protiens      Albumin
## Min.   : 10.0      Min.   :2.700    Min.   :0.900
## 1st Qu.: 25.0      1st Qu.:5.800    1st Qu.:2.600
## Median : 42.0      Median :6.600    Median :3.100
## Mean   :109.9      Mean   :6.483    Mean   :3.142
## 3rd Qu.: 87.0      3rd Qu.:7.200    3rd Qu.:3.800
## Max.   :4929.0     Max.   :9.600    Max.   :5.500
##
## Albumin_and_Globulin_Ratio      target
## Min.   :0.3000      Min.   :0.0000
## 1st Qu.:0.7000      1st Qu.:0.0000
## Median :0.9300      Median :1.0000
## Mean   :0.9471      Mean   :0.7136
## 3rd Qu.:1.1000      3rd Qu.:1.0000
## Max.   :2.8000      Max.   :1.0000
## NAs      :4

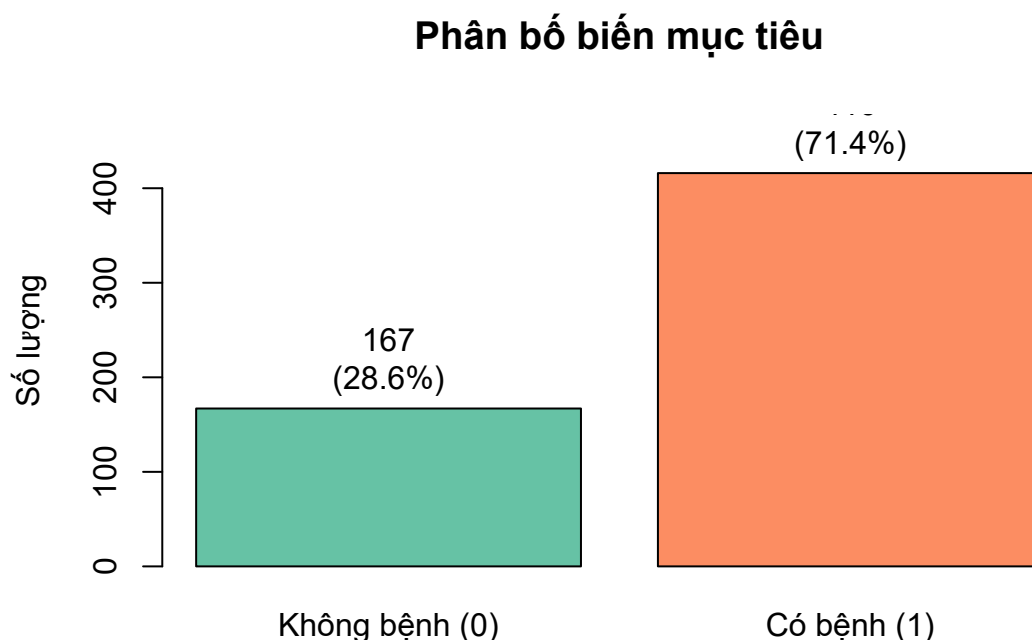
colSums(is.na(df))

##      Age      Gender
##      0      0
##      Total_Bilirubin      Direct_Bilirubin
##      0      0
##      Alkaline_Phosphotase      Alamine_Aminotransferase
##      0      0
##      Aspartate_Aminotransferase      Total_Protiens
##      0      0
##      Albumin      Albumin_and_Globulin_Ratio
##      0      4
##      target
##      0
```

Toàn bộ dữ liệu chỉ có 4 giá trị khuyết, đều nằm ở biến `Albumin_and_Globulin_Ratio`. Con số này rất nhỏ so với 583 quan sát nên cách xử lý gần như không ảnh hưởng tới kết quả. Dù vậy, các giá trị khuyết vẫn được điền bằng trung vị thay vì loại bỏ quan sát, và điều quan trọng hơn là việc điền khuyết được đặt bên trong quy trình huấn luyện để trung vị chỉ được tính từ tập huấn luyện. Lý do của yêu cầu này được trình bày ở Mục 5.

4 Phân tích khám phá dữ liệu

4.1 Phân bố biến mục tiêu



Hình 1: Phân bố biến mục tiêu

Tỉ lệ 71,4% và 28,6% cho thấy dữ liệu mất cân bằng ở mức trung bình. Hệ quả trực tiếp là độ chính xác tổng thể trở thành một chỉ số gây hiểu nhầm, bởi một mô hình dự đoán tất cả bệnh nhân đều mắc bệnh đã đạt độ chính xác 0,714 mà không mang lại thông tin nào. Vì vậy toàn bộ phần đánh giá về sau sử dụng recall và precision của từng lớp cùng với macro F1, thay vì dựa vào độ chính xác tổng thể.

4.2 Tương quan giữa các biến

Ba cặp biến có tương quan rất cao. Cặp thứ nhất là Total_Bilirubin với Direct_Bilirubin ($r = 0,87$), cặp thứ hai là hai men gan Alamine_Aminotransferase và Aspartate_Aminotransferase ($r = 0,79$), cặp thứ ba là Total_Protiens với Albumin ($r = 0,78$). Các quan hệ này phù hợp với bản chất sinh hoá của những chỉ số đó, khi bilirubin trực tiếp là một thành phần của bilirubin toàn phần và albumin là thành phần chính của protein toàn phần. Tuy nhiên chúng lại gây khó khăn cho hồi quy logistic, bởi các biến

Ma trận tương quan

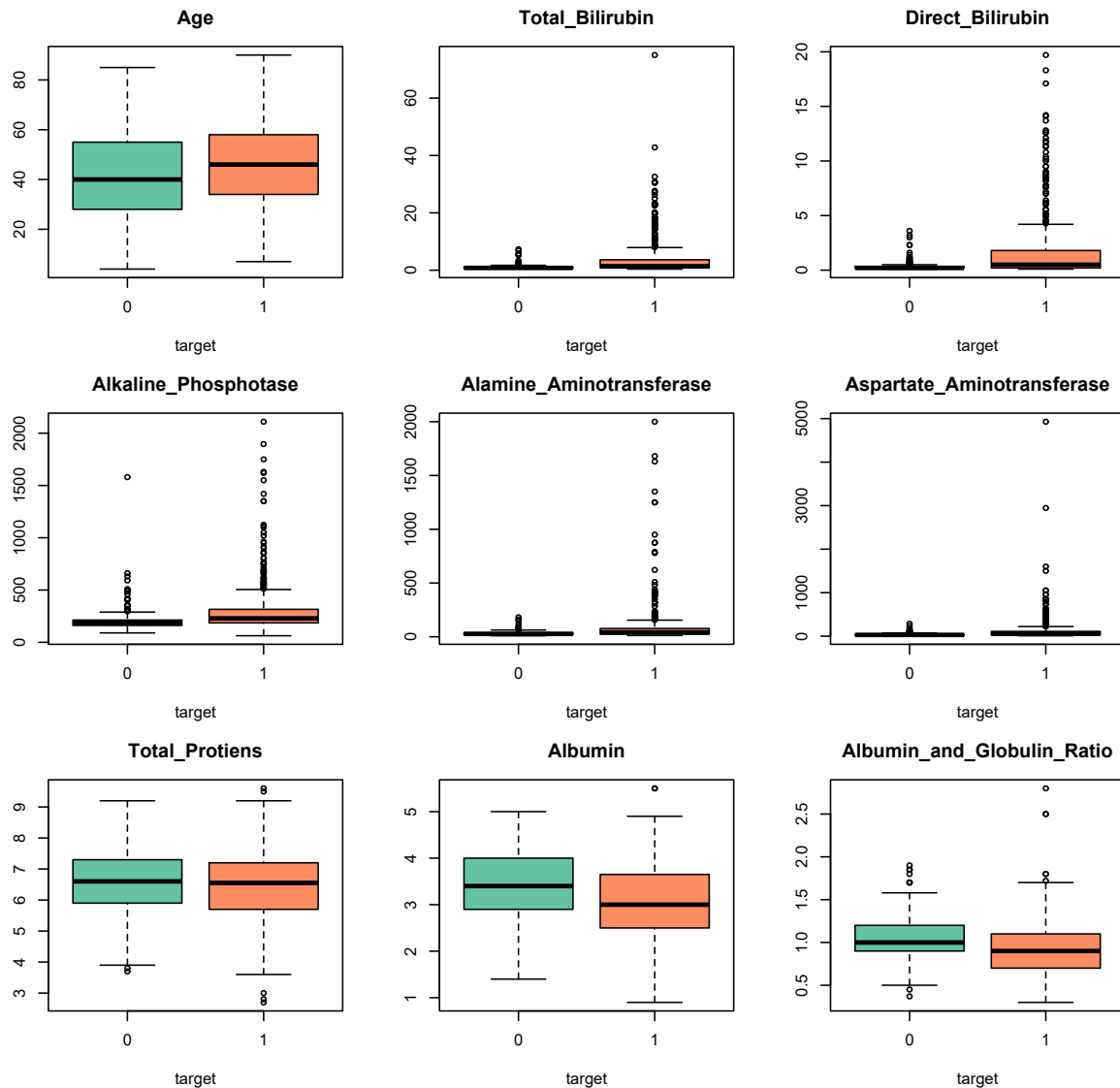
Age	1.00	0.01	0.01	0.08	-0.09	-0.02	-0.19	-0.26	-0.22	0.13
Total_Bilirubin	0.01	1.00	0.87	0.21	0.21	0.24	-0.01	-0.22	-0.21	0.22
Direct_Bilirubin	0.01	0.87	1.00	0.23	0.23	0.26	0.00	-0.23	-0.20	0.25
Alkaline_Phosphotase	0.08	0.21	0.23	1.00	0.12	0.17	-0.03	-0.16	-0.23	0.18
Alamine_Aminotransferase	-0.09	0.21	0.23	0.12	1.00	0.79	-0.04	-0.03	-0.00	0.16
Aspartate_Aminotransferase	-0.02	0.24	0.26	0.17	0.79	1.00	-0.03	-0.08	-0.07	0.15
Total_Protiens	-0.19	-0.01	0.00	-0.03	-0.04	-0.03	1.00	0.78	0.23	-0.03
Albumin	-0.26	-0.22	-0.23	-0.16	-0.03	-0.08	0.78	1.00	0.69	-0.16
Albumin_and_Globulin_Ratio	-0.22	-0.21	-0.20	-0.23	-0.00	-0.07	0.23	0.69	1.00	-0.16
target	0.13	0.22	0.25	0.18	0.16	0.15	-0.03	-0.16	-0.16	1.00
	Age	Total_Bilirubin	Direct_Bilirubin	Alkaline_Phosphotase	Alamine_Aminotransferase	Aspartate_Aminotransferase	Total_Protiens	Albumin	Albumin_and_Globulin_Ratio	target

Hình 2: Ma trận tương quan giữa các biến sinh hoá và biến mục tiêu

gần như trùng thông tin sẽ làm ma trận thiết kế gần suy biến và khiến hệ số ước lượng mất ổn định.

Ở chiều ngược lại, tương quan giữa từng biến với biến mục tiêu đều yếu, giá trị lớn nhất chỉ đạt 0,25. Điều này cho thấy không có biến đơn lẻ nào tách được hai nhóm một cách rõ ràng.

4.3 Phân bố các biến sinh hoá và biến đổi log



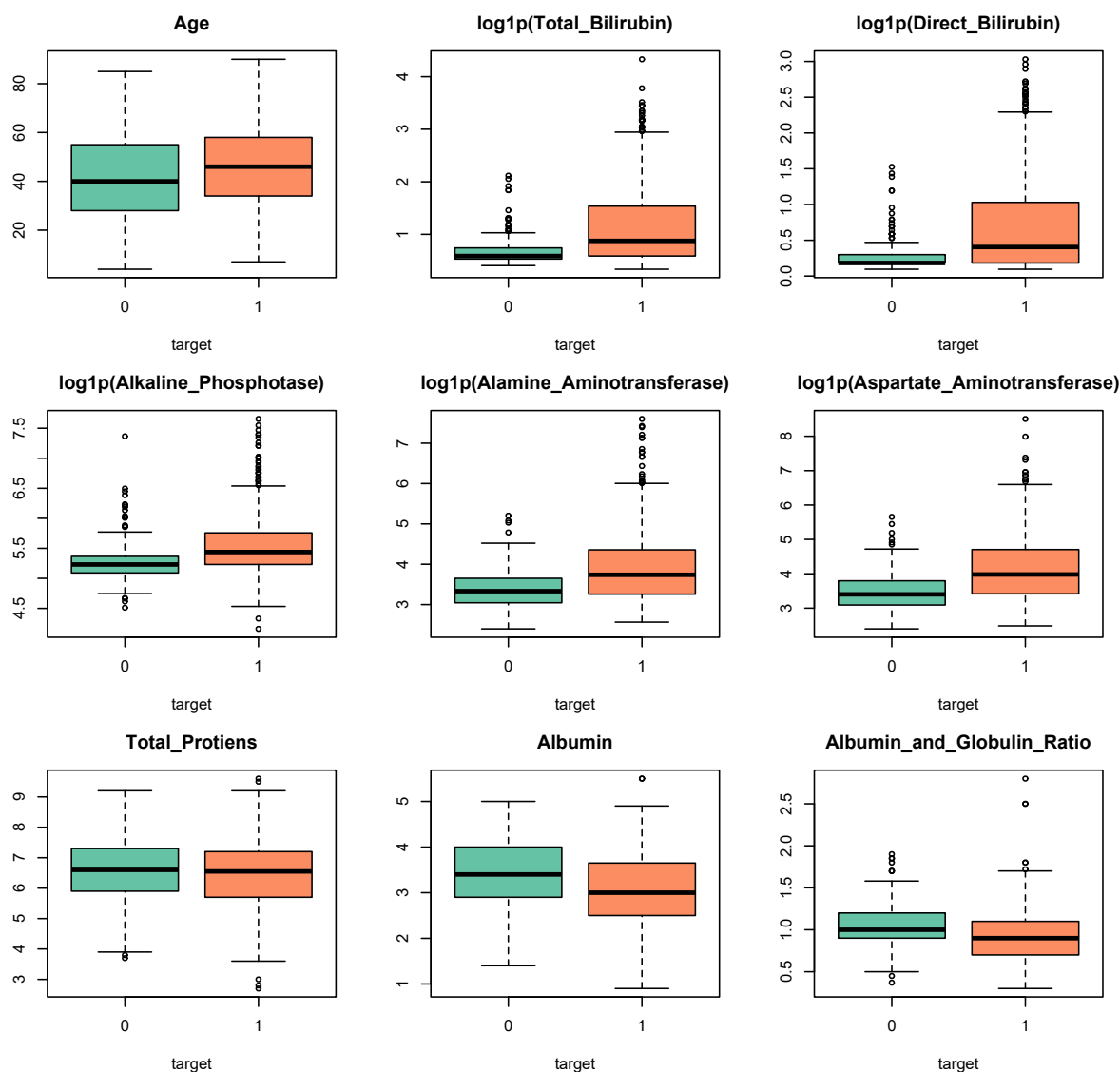
Hình 3: Phân bố các biến sinh hoá theo nhóm bệnh trên thang đo gốc

Hình 3 cho thấy năm biến gồm Total_Bilirubin, Direct_Bilirubin, Alkaline_Phosphotase, Alamine_Aminotransferase và Aspartate_Aminotransferase đều có phân phối lệch phải rõ rệt. Phần lớn các quan sát tập trung ở vùng giá trị thấp, trong khi một số ít giá trị rất lớn tạo thành đuôi phân phối kéo dài và nhiều điểm ngoại lai. Do đó, phần hộp của boxplot bị nén sát phía dưới, gây khó khăn cho việc quan sát và so sánh phân bố giữa hai nhóm bệnh.

Để giảm mức độ bất đối xứng, phép biến đổi $\log(1 + x)$ được áp dụng cho năm biến trên. Vì $\log(1 + x)$ là một hàm đơn điệu tăng nên phép biến đổi này bảo toàn thứ tự giữa các quan sát, đồng thời làm giảm ảnh hưởng của các giá trị lớn bằng cách nén phần đuôi của phân phối.

Hiệu quả của phép biến đổi được thể hiện ở Hình 4. Sau khi biến đổi log, các phân phối trở nên cân đối hơn, phần hộp của boxplot được trải rộng thay vì

tập trung sát đáy, trong khi các giá trị ngoại lai không còn chi phối mạnh thang đo của biểu đồ. Nhờ đó, sự khác biệt về vị trí trung tâm và mức độ phân tán giữa hai nhóm bệnh được thể hiện trực quan hơn, tạo điều kiện thuận lợi cho các bước phân tích và xây dựng mô hình ở các phần tiếp theo.



Hình 4: Phân bố các biến sinh hoá theo nhóm bệnh sau biến đổi log

4.4 Đa cộng tuyến và hệ số VIF

Hệ số tương quan chỉ phản ánh mối quan hệ giữa từng cặp biến, trong khi hiện tượng đa cộng tuyến có thể phát sinh từ mối quan hệ đồng thời giữa một biến với nhiều biến giải thích khác. Vì vậy, báo cáo sử dụng hệ số phóng đại phương sai (Variance Inflation Factor – VIF) để đánh giá mức độ đa cộng tuyến của từng

biến. Hệ số VIF được xác định theo công thức

$$\text{VIF}_j = \frac{1}{1 - R_j^2},$$

trong đó R_j^2 là hệ số xác định của mô hình hồi quy biến thứ j theo tất cả các biến còn lại. Theo quy ước thường được sử dụng, giá trị VIF lớn hơn 5 cho thấy dấu hiệu đa cộng tuyến đáng lưu ý, trong khi VIF lớn hơn 10 được xem là nghiêm trọng.

Khác với các kiểm định dựa trên thứ hạng, hệ số VIF không bất biến đối với các phép biến đổi đơn điệu như phép biến đổi log. Do mô hình trong nghiên cứu được xây dựng trên dữ liệu sau biến đổi log, VIF được tính trên cả thang đo gốc và thang đo log nhằm đánh giá ảnh hưởng của phép biến đổi đến mức độ đa cộng tuyến.

```
compute_vif <- function(data) {  
  vars <- colnames(data)  
  sapply(vars, function(v) {  
    others <- setdiff(vars, v)  
    fml <- as.formula(paste(v, "~", paste(others, collapse = " + ")))  
    r2 <- summary(lm(fml, data = data))$r.squared  
    1 / (1 - r2)  
  })  
}  
  
# Thang gốc: dùng df_raw (bản trước biến đổi Log)  
X_vif <- df_raw[, c(numeric_cols, "Gender")]  
X_vif$Gender <- ifelse(X_vif$Gender == "Male", 1, 0)  
X_vif$Albumin_and_Globulin_Ratio[is.na(X_vif$Albumin_and_Globulin_Ratio)] <-  
  median(X_vif$Albumin_and_Globulin_Ratio, na.rm = TRUE)  
vif_raw <- compute_vif(X_vif)  
  
# Thang log: log1p cho các biến lệch phải  
X_vif_log <- X_vif  
for (col in skewed_cols) X_vif_log[[col]] <- log1p(df_raw[[col]])  
vif_log <- compute_vif(X_vif_log)  
  
vif_tbl <- data.frame(  
  Bien = names(vif_raw),  
  VIF_goc = round(vif_raw, 2),  
  VIF_log = round(vif_log[names(vif_raw)], 2),  
  chênh_lech = round(vif_log[names(vif_raw)] - vif_raw, 2),  
  row.names = NULL  
)  
vif_tbl <- vif_tbl[order(-vif_tbl$VIF_log), ]
```

Bảng 1: Hệ số VIF trên thang đo gốc và thang đo log

Biến	VIF gốc	VIF log	Chênh lệch
Total_Bilirubin	4.28	20.72	16.44
Direct_Bilirubin	4.53	20.55	16.03
Albumin	10.12	10.62	0.49
Total_Protiens	5.55	5.65	0.10
Aspartate_Aminotransferase	2.81	4.18	1.37
Alamine_Aminotransferase	2.82	3.94	1.12
Albumin_and_Globulin_Ratio	3.68	3.75	0.07
Alkaline_Phosphotase	1.12	1.28	0.16
Age	1.10	1.10	0.00
Gender	1.03	1.06	0.03

Kết quả cho thấy phép biến đổi log ảnh hưởng đáng kể đến mức độ đa cộng tuyến của một số biến. Đáng chú ý nhất là hai biến Total_Bilirubin và Direct_Bilirubin, với giá trị VIF tăng từ khoảng 4,3 trên thang đo gốc lên trên 20 sau khi biến đổi log. Điều này cho thấy mối quan hệ tuyến tính giữa hai biến trở nên mạnh hơn trên thang đo log, dẫn đến hiện tượng đa cộng tuyến nghiêm trọng.

Đối với các biến còn lại, giá trị VIF thay đổi không đáng kể sau phép biến đổi. Riêng biến Albumin có VIF xấp xỉ 10 trên cả hai thang đo, cho thấy hiện tượng đa cộng tuyến đã tồn tại ngay từ dữ liệu gốc và hầu như không chịu ảnh hưởng của phép biến đổi log. Các biến khác đều có VIF nhỏ hơn 6, cho thấy mức độ đa cộng tuyến không đáng kể.

Do mô hình được xây dựng trên dữ liệu sau biến đổi log, kết quả VIF trên thang đo này được sử dụng để lựa chọn đặc trưng. Trong đó, Direct_Bilirubin được loại bỏ do có mức đa cộng tuyến rất cao với Total_Bilirubin, trong khi Total_Bilirubin được giữ lại vì mang ý nghĩa lâm sàng tổng quát hơn.

4.5 Thống kê mô tả theo nhóm

Bảng thống kê dưới đây được tính trên thang đo gốc, để các con số giữ nguyên đơn vị lâm sàng và có thể đối chiếu với khoảng tham chiếu xét nghiệm.

```
group_stats <- do.call(rbind, lapply(numeric_cols, function(col) {
  s <- tapply(df_raw[[col]], df_raw$target, function(x) {
    x <- x[!is.na(x)]
    c(Min = min(x), Max = max(x), Mean = mean(x),
      Median = median(x), SD = sd(x))
  })
  data.frame(Bien = col, Nhóm = c("Không bệnh (0)", "Có bệnh (1)"),
    round(do.call(rbind, s), 2), row.names = NULL)
}))
```


Bảng 2: Thống kê mô tả các biến sinh hoá theo nhóm bệnh (thang đo gốc)

Biến	Nhóm	Min	Max	Mean	Median	SD
Age	Không bệnh (0)	4.00	85.0	41.24	40.00	17.00
Age	Có bệnh (1)	7.00	90.0	46.15	46.00	15.65
Total_Bilirubin	Không bệnh (0)	0.50	7.3	1.14	0.80	1.00
Total_Bilirubin	Có bệnh (1)	0.40	75.0	4.16	1.40	7.14
Direct_Bilirubin	Không bệnh (0)	0.10	3.6	0.40	0.20	0.52
Direct_Bilirubin	Có bệnh (1)	0.10	19.7	1.92	0.50	3.21
Alkaline_Phosphotase	Không bệnh (0)	90.00	1580.0	219.75	186.00	140.99
Alkaline_Phosphotase	Có bệnh (1)	63.00	2110.0	319.01	229.00	268.31
Alamine_Aminotransferase	Không bệnh (0)	10.00	181.0	33.65	27.00	25.06
Alamine_Aminotransferase	Có bệnh (1)	12.00	2000.0	99.61	41.00	212.77
Aspartate_Aminotransferase	Không bệnh (0)	10.00	285.0	40.69	29.00	36.41
Aspartate_Aminotransferase	Có bệnh (1)	11.00	4929.0	137.70	52.50	337.39
Total_Protiens	Không bệnh (0)	3.70	9.2	6.54	6.60	1.06
Total_Protiens	Có bệnh (1)	2.70	9.6	6.46	6.55	1.09
Albumin	Không bệnh (0)	1.40	5.0	3.34	3.40	0.78
Albumin	Có bệnh (1)	0.90	5.5	3.06	3.00	0.79
Albumin_and_Globulin_Ratio	Không bệnh (0)	0.37	1.9	1.03	1.00	0.29
Albumin_and_Globulin_Ratio	Có bệnh (1)	0.30	2.8	0.91	0.90	0.33

Bảng thống kê mô tả cho thấy nhóm bệnh nhân mắc bệnh gan (target = 1) có xu hướng lớn tuổi hơn so với nhóm không mắc bệnh, với tuổi trung bình lần lượt là 46,15 và 41,24.

Đối với các chỉ số bilirubin và men gan, nhóm mắc bệnh có giá trị trung bình, trung vị và độ phân tán đều cao hơn rõ rệt. Cụ thể, Total_Bilirubin tăng từ 1,14 lên 4,16 mg/dL, Direct_Bilirubin tăng từ 0,40 lên 1,92 mg/dL, trong khi các men gan Alamine_Aminotransferase, Aspartate_Aminotransferase và Alkaline_Phosphotase đều có giá trị trung bình cao hơn đáng kể ở nhóm mắc bệnh. Đồng thời, độ lệch chuẩn của các biến này cũng lớn hơn nhiều, cho thấy mức độ biến thiên cao và sự xuất hiện của các giá trị rất lớn trong nhóm bệnh.

Ngược lại, các chỉ số phản ánh chức năng tổng hợp của gan có xu hướng giảm ở nhóm mắc bệnh. Giá trị trung bình của Albumin giảm từ 3,34 xuống 3,06 g/dL, trong khi tỷ số Albumin_and_Globulin_Ratio giảm từ 1,03 xuống 0,91. Hai nhóm có giá trị Total_Proteins tương đối tương đồng, với trung bình lần lượt là 6,54 và 6,46 g/dL.

Nhìn chung, các thống kê mô tả cho thấy nhóm mắc bệnh có xu hướng tăng các chỉ số bilirubin và men gan, đồng thời giảm nồng độ albumin và tỷ số albumin/globulin.

4.6 Lựa chọn đặc trưng

Mục tiêu của bước này là xác định những đặc trưng có mối liên hệ với biến mục tiêu - tức tình trạng mắc bệnh gan. Quá trình thực hiện kiểm định giả thuyết được xem như một bước sàng lọc ban đầu nhằm loại bỏ các biến không có bằng chứng thống kê về mối liên hệ với biến mục tiêu. Ta tiến hành kiểm định với cặp giả thuyết sau:

- H_0 : Biến không có mối liên hệ với tình trạng mắc bệnh gan (phân phối của biến là như nhau giữa hai nhóm đối với biến liên tục, hoặc độc lập với biến mục tiêu đối với biến phân loại).
- H_1 : Biến có mối liên hệ với tình trạng mắc bệnh gan.

Do hầu hết các biến liên tục có phân phối lệch, sự khác biệt giữa hai nhóm được đánh giá bằng kiểm định phi tham số Mann–Whitney U. Riêng biến Gender là biến phân loại nên được kiểm định bằng phép kiểm Chi-square. Để kiểm soát sai số do thực hiện nhiều phép kiểm đồng thời, các giá trị p được hiệu chỉnh bằng thủ tục Benjamini–Hochberg nhằm kiểm soát tỷ lệ phát hiện sai (False Discovery Rate – FDR).

```
fs <- data.frame(feature = character(), p_value = numeric(),
                 effect_size = numeric(), stringsAsFactors = FALSE)

for (col in numeric_cols) {
  g0 <- df[[col]][df$target == 0]
  g1 <- df[[col]][df$target == 1]
  w <- wilcox.test(g0, g1, exact = FALSE)
  n0 <- sum(!is.na(g0)); n1 <- sum(!is.na(g1))
  rb <- 1 - 2 * w$statistic / (n0 * n1) # rank-biserial correlation
  fs <- rbind(fs, data.frame(feature = col, p_value = w$p.value,
                           effect_size = as.numeric(rb)))
}

# Gender là biến phân loại: dùng Chi-square, kích thước ảnh hưởng Cramer's V
contingency <- table(df$Gender, df$target)
chi_res <- chisq.test(contingency)
n_total <- sum(contingency)
cramers_v <- sqrt(chi_res$statistic /
                 (n_total * (min(dim(contingency)) - 1)))
fs <- rbind(fs, data.frame(feature = "Gender", p_value = chi_res$p.value,
                         effect_size = as.numeric(cramers_v)))

fs$p_fdr <- p.adjust(fs$p_value, method = "BH") # Benjamini-Hochberg
fs$significant <- fs$p_fdr < 0.05
fs <- fs[order(fs$p_value), ]

fs_disp <- fs
fs_disp$p_value <- signif(fs_disp$p_value, 3)
fs_disp$effect_size <- round(fs_disp$effect_size, 3)
fs_disp$p_fdr <- signif(fs_disp$p_fdr, 3)
```

Bảng 3: Kiểm định lựa chọn đặc trưng (Mann–Whitney U / Chi-square, hiệu chỉnh FDR)

Biến	p-value	Kích thước ảnh hưởng	p (FDR)	Có ý nghĩa
Aspartate_Aminotransferase	0.00e+00	0.394	0.00e+00	TRUE
Total_Bilirubin	0.00e+00	0.387	0.00e+00	TRUE
Direct_Bilirubin	0.00e+00	0.372	0.00e+00	TRUE
Alamine_Aminotransferase	0.00e+00	0.371	0.00e+00	TRUE
Alkaline_Phosphotase	0.00e+00	0.349	0.00e+00	TRUE
Albumin_and_Globulin_Ratio	5.80e-06	-0.240	9.70e-06	TRUE
Albumin	5.57e-05	-0.213	7.95e-05	TRUE
Age	1.77e-03	0.165	2.22e-03	TRUE
Gender	5.97e-02	0.078	6.63e-02	FALSE
Total_Protiens	4.37e-01	-0.041	4.37e-01	FALSE

Kết quả cho thấy 8 trong số 10 biến vẫn đạt ý nghĩa thống kê sau hiệu chỉnh FDR. Hai biến không đạt mức ý nghĩa là Gender ($p_{FDR} = 0,066$) và Total_Proteins ($p_{FDR} = 0,437$), do đó được loại khỏi mô hình. Trong các biến còn lại, nhóm chỉ số bilirubin và men gan có kích thước ảnh hưởng lớn nhất (effect size khoảng 0,35–0,39), cho thấy khả năng phân biệt tốt giữa hai nhóm bệnh.

5 Thiết kế và lựa chọn mô hình

Tập biến sử dụng

Trên cơ sở kết quả phân tích khám phá và lựa chọn đặc trưng, bảy biến được đưa vào mô hình, gồm Age, Total_Bilirubin, Alkaline_Phosphotase, Alamine_Aminotransferase, Aspartate_Aminotransferase, Albumin và Albumin_and_Globulin_Ratio. Trong số này, bốn biến có phân phối lệch phải được sử dụng sau phép biến đổi $\log(1 + x)$ nhằm hạn chế ảnh hưởng của các giá trị quá lớn. Biến Direct_Bilirubin được loại do đa cộng tuyến mạnh với Total_Bilirubin; hai biến Gender và Total_Protiens không đạt ý nghĩa thống kê sau hiệu chỉnh FDR nên không được đưa vào mô hình.

Chia dữ liệu và phòng ngừa rò rỉ

Toàn bộ dữ liệu được chia phân tầng theo tỷ lệ 80:20 để tỷ lệ hai lớp trong mỗi tập gần với tỷ lệ của dữ liệu ban đầu. Tập huấn luyện gồm 466 quan sát, trong đó có 333 trường hợp mắc bệnh; tập kiểm tra gồm 117 quan sát, với 83 trường hợp mắc bệnh. Tập kiểm tra chỉ được sử dụng sau khi quá trình lựa chọn mô hình và ngưỡng phân loại đã hoàn tất.

Để tránh rò rỉ dữ liệu, các bước điều chỉnh, chuẩn hóa và cân bằng lớp đều được thực hiện trong phạm vi dữ liệu huấn luyện của từng fold. Trung vị và

các tham số chuẩn hóa vì thế không mang thông tin từ phần xác thực. Tương tự, SMOTE chỉ tạo mẫu từ phần huấn luyện trước khi mô hình được ước lượng, nhờ đó kết quả đánh giá phản ánh đúng khả năng dự đoán trên dữ liệu chưa xuất hiện trong quá trình học.

Mô hình và chỉ số đánh giá

Hồi quy logistic được lựa chọn vì phù hợp với bài toán phân loại nhị phân và cho phép ước lượng trực tiếp xác suất mắc bệnh. Đặc điểm này cũng tạo cơ sở cho việc điều chỉnh ngưỡng phân loại theo mục tiêu sàng lọc ở phần tiếp theo. SMOTE được kết hợp trong quá trình huấn luyện để hạn chế ảnh hưởng của sự chênh lệch số lượng giữa hai lớp.

Vì lớp có bệnh chiếm đa số, Accuracy có thể che khuất khả năng nhận diện của từng nhóm. Do đó, kết quả được xem xét thông qua recall và precision của cả lớp có bệnh (lớp 1) và lớp không bệnh (lớp 0). Macro F1 được dùng làm tiêu chí lựa chọn mô hình vì chỉ số này dành trọng số ngang nhau cho hai lớp, qua đó đưa ra đánh giá cân bằng hơn về chất lượng phân loại tổng thể.

Lựa chọn mô hình bằng kiểm định chéo

Trong nghiên cứu này, mỗi phương án là một cấu hình của mô hình hồi quy logistic, được xác định bởi ba lựa chọn: dạng phạt L1 (Lasso) hoặc L2 (Ridge); hệ số $C \in \{0,001; 0,01; 0,1; 1; 10; 100\}$ kiểm soát mức điều chuẩn (regularization); và số láng giềng của SMOTE $k \in \{3; 5; 7\}$. Sự kết hợp của các giá trị này tạo thành 36 cấu hình. Các cấu hình sử dụng cùng tập biến và các bước tiền xử lý, chỉ khác nhau ở ba lựa chọn trên.

Những cấu hình trên được đánh giá trên tập huấn luyện bằng kiểm định chéo phân tầng 5 fold (5-fold stratified cross-validation). Ở mỗi lượt, mô hình được huấn luyện trên bốn fold và đánh giá trên fold còn lại. Hiệu năng của mỗi phương án được tổng hợp thông qua Macro F1 trung bình trên năm fold. Phương án đạt giá trị Macro F1 trung bình cao nhất được lựa chọn và huấn luyện lại trên toàn bộ tập huấn luyện để thu được mô hình cuối cùng.

Ở bảng dưới đây, cấu hình đạt kết quả cao nhất sử dụng phạt L2, $C = 1$ và SMOTE với $k = 3$, cho Macro F1 trung bình bằng 0,6279.

Bảng 4: Năm cấu hình có Macro F1 cao nhất trong kiểm định chéo 5 fold

Hạng	C	Dạng phạt	SMOTE k	Macro F1	SD
1	1.0	L2	3	0.6279	0.0385
2	0.1	L2	3	0.6274	0.0346
3	0.1	L2	5	0.6261	0.0366
4	100.0	L2	3	0.6259	0.0369
4	100.0	L1	3	0.6259	0.0369

6 Lựa chọn ngưỡng phân loại

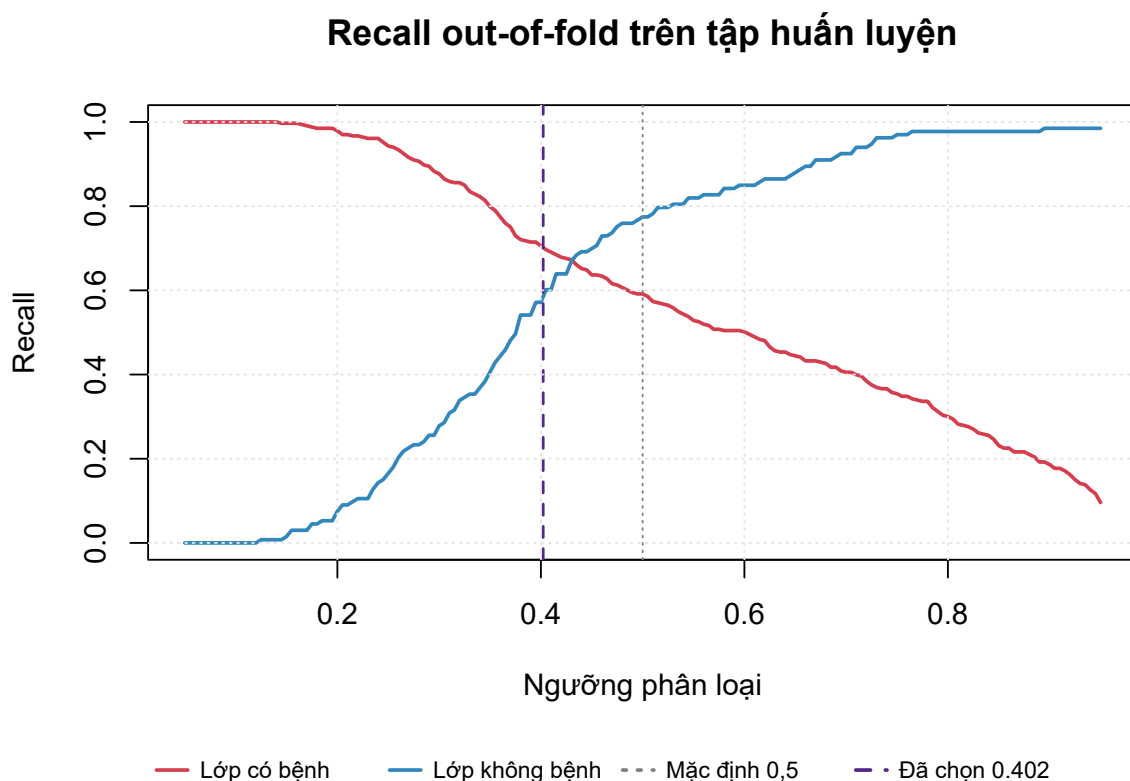
Việc khảo sát ngưỡng tập trung trước hết vào recall của lớp có bệnh. Chỉ số này là tỷ lệ ca bệnh được phát hiện trên tổng số ca bệnh thực tế, nên phản ánh trực tiếp mức độ bỏ sót bệnh nhân. Trong bối cảnh hỗ trợ sàng lọc, âm tính giả cần được ưu tiên kiểm soát vì có thể khiến một ca bệnh không được xem xét kịp thời; trong khi cảnh báo nhầm vẫn có thể được kiểm tra lại bằng các bước xác nhận tiếp theo. Accuracy không thể hiện riêng loại sai sót này.

Nếu hạ ngưỡng quá thấp, mô hình có thể báo bệnh cho gần như mọi quan sát và mất khả năng nhận diện người không bệnh. Vì vậy, recall lớp không bệnh được theo dõi song song; ngưỡng cao nhất còn đáp ứng recall mục tiêu được chọn để vừa giảm bỏ sót, vừa hạn chế số cảnh báo nhầm tăng thêm.

Sau khi xác định mô hình phù hợp, ngưỡng phân loại được lựa chọn dựa trên xác suất dự đoán out-of-fold của tập huấn luyện, thu được từ cùng quy trình kiểm định chéo phân tầng 5 fold. Do mục tiêu của bài toán là ưu tiên phát hiện các trường hợp mắc bệnh, ngưỡng phân loại được xác định sao cho **Recall của lớp bệnh đạt tối thiểu 70%** nhằm duy trì khả năng phát hiện bệnh cần thiết, đồng thời không làm gia tăng quá mức số trường hợp không mắc bệnh bị cảnh báo nhầm.

Bảng 5: Ngưỡng cao nhất đáp ứng từng mức recall mục tiêu trên tập huấn luyện

Recall mục tiêu	Ngưỡng	Recall lớp 1	Recall lớp 0
70%	0.402	0.703	0.586
75%	0.372	0.751	0.489
80%	0.350	0.802	0.406
85%	0.324	0.853	0.338
90%	0.283	0.901	0.241
95%	0.246	0.952	0.150



Hình 5: Đánh đổi giữa recall của hai lớp theo ngưỡng phân loại

Ngưỡng được chọn là 0,402. Trên dự đoán out-of-fold của tập huấn luyện, ngưỡng này cho recall lớp có bệnh 0,703 và recall lớp không bệnh 0,586. Bảng 5 cũng cho thấy nếu nâng mục tiêu recall từ 70% lên 80%, ngưỡng phải giảm còn khoảng 0,350 và recall của lớp không bệnh giảm xuống 0,406. Vì vậy, mức mục tiêu 70% tạo ra điểm khởi đầu thận trọng hơn cho bài toán sàng lọc, thay vì đánh đổi quá mạnh độ đặc hiệu chỉ để tối đa hóa độ nhạy.

7 Đánh giá trên tập kiểm tra

Mô hình tốt nhất được huấn luyện lại trên toàn bộ 466 quan sát huấn luyện, sau đó đánh giá đúng một lần trên 117 quan sát kiểm tra. Ở ngưỡng 0,402, ma trận nhầm lẫn gồm 24 trường hợp không bệnh được nhận diện đúng, 10 báo động nhầm, 14 ca bệnh bị bỏ sót và 69 ca bệnh được phát hiện.

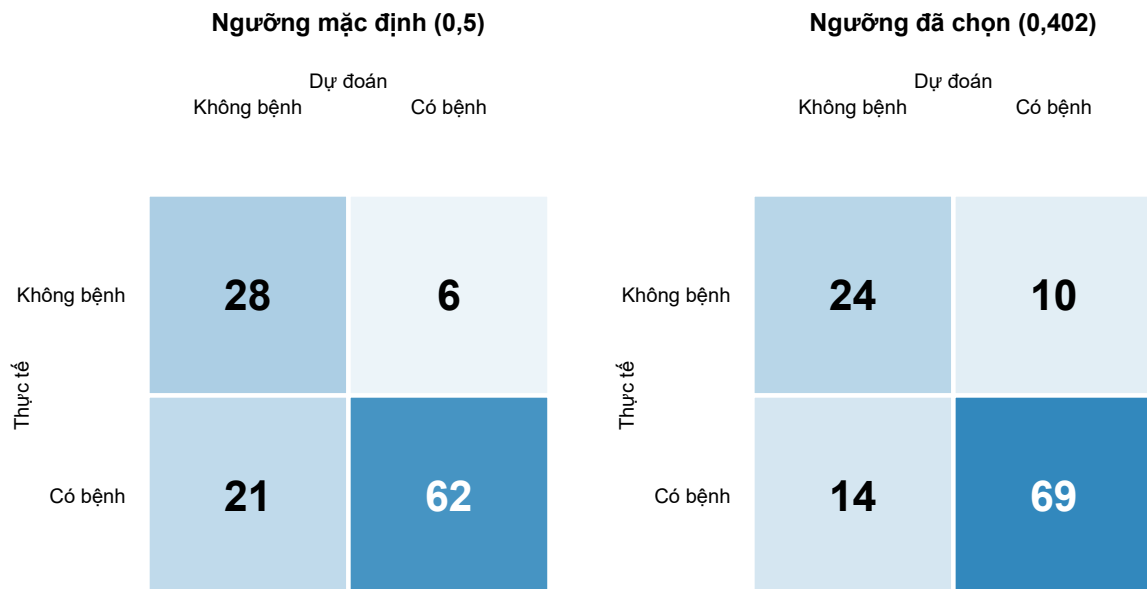
##	Dự đoán		
##	Thực tế	Không bệnh	Có bệnh
##	Không bệnh	24	10
##	Có bệnh	14	69

Bảng 6: Các chỉ số trên tập kiểm tra với ngưỡng 0,402

Accuracy	Recall lớp 1	Precision lớp 1	Recall lớp 0	Precision lớp 0	Macro F1
0.795	0.831	0.873	0.706	0.632	0.759

Bảng 7: So sánh ngưỡng mặc định và ngưỡng được chọn trên tập kiểm tra

Ngưỡng	Accuracy	Recall lớp 1	Precision lớp 1	Recall lớp 0	Precision lớp 0	Macro F1
0,500 (mặc định)	0.769	0.747	0.912	0.824	0.571	0.748
0,402 (đã chọn)	0.795	0.831	0.873	0.706	0.632	0.759



Hình 6: Ma trận nhầm lẫn trên tập kiểm tra ở hai ngưỡng phân loại

So với ngưỡng mặc định, ngưỡng 0,402 làm số âm tính giả giảm từ 21 xuống 14, nghĩa là phát hiện thêm bảy ca bệnh. Số dương tính giả tăng từ 6 lên 10, nhưng mức tăng này nhỏ hơn số dự đoán đúng thu hồi được ở lớp có bệnh. Do đó Accuracy tăng từ 0,769 lên 0,795 và Macro F1 tăng từ 0,748 lên 0,759, thay vì phải hy sinh hiệu năng tổng thể để đổi lấy độ nhạy.

Recall của lớp có bệnh tăng từ 0,747 lên 0,831, trong khi recall của lớp không bệnh giảm từ 0,824 xuống 0,706. Precision lớp có bệnh vẫn ở mức cao, 0,873, cho thấy phần lớn cảnh báo dương tính là đúng trong tập kiểm tra này. Đồng thời, precision của lớp không bệnh tăng từ 0,571 lên 0,632 vì số ca bệnh bị xếp nhầm vào nhóm âm tính đã giảm rõ rệt. Như vậy, ngưỡng mới tạo ra một ma trận sai số cân đối hơn và phù hợp hơn với mục tiêu hạn chế bỏ sót ca bệnh.

8 Khoảng tin cậy Bootstrap

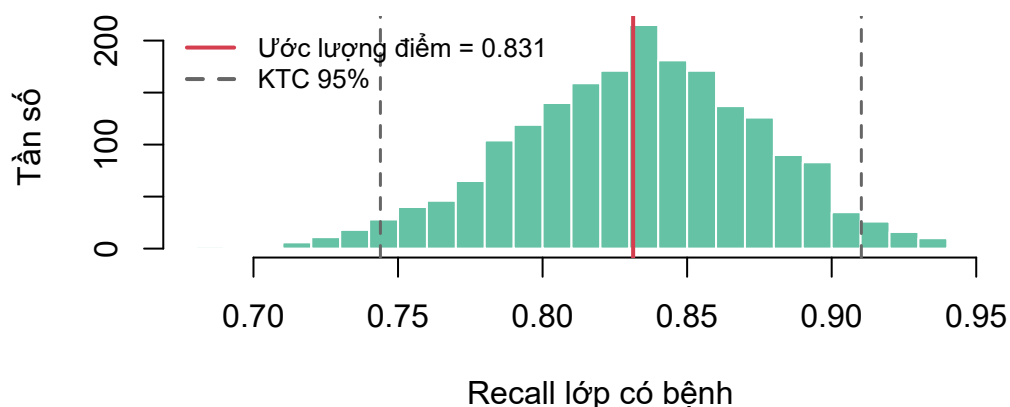
Do các chỉ số đánh giá được ước lượng từ một tập kiểm tra có kích thước hữu hạn, phương pháp Bootstrap được sử dụng để ước lượng khoảng tin cậy của các kết quả. Cụ thể, 2.000 mẫu Bootstrap được tạo bằng cách lấy mẫu có hoàn lại từ các cặp gồm nhãn thực tế và dự đoán cuối cùng, với kích thước mỗi mẫu được giữ nguyên là 117 quan sát.

Trong quá trình này, mô hình không được huấn luyện lại ở từng lần lấy mẫu. Vì vậy, khoảng tin cậy thu được phản ánh sự biến thiên từ các thành phần của tập kiểm tra và không bao gồm bất định phát sinh từ quá trình huấn luyện trên các tập dữ liệu khác nhau.

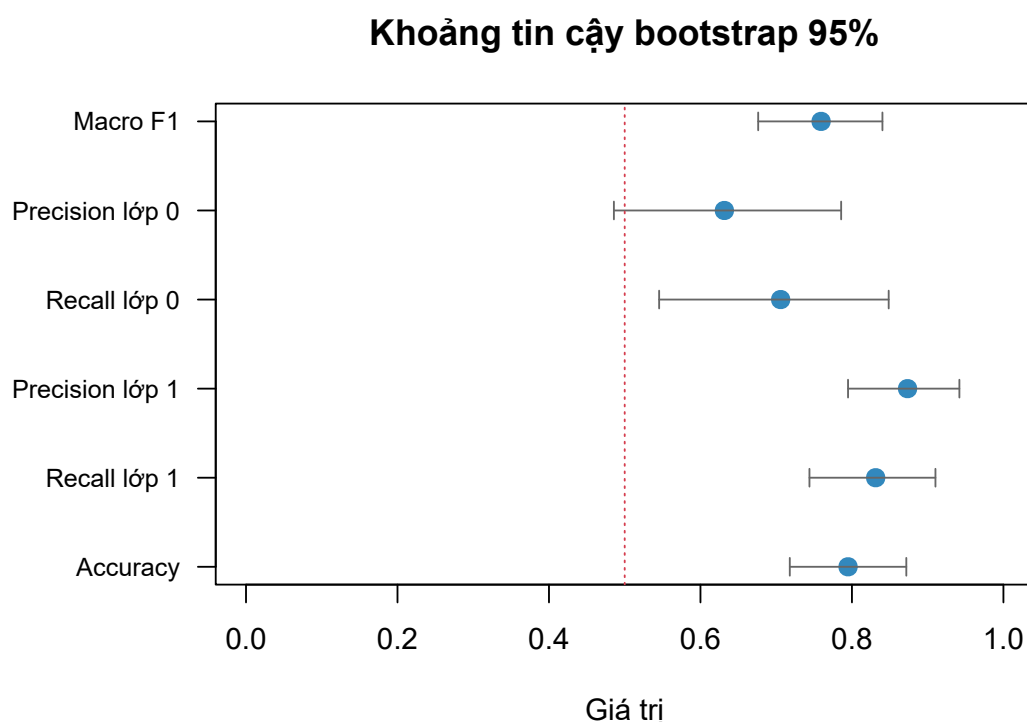
Bảng 8: Khoảng tin cậy bootstrap 95% cho các chỉ số trên tập kiểm tra

Chỉ số	Ước lượng điểm	Cận dưới	Cận trên	Sai số chuẩn
Accuracy	0.7949	0.7179	0.8718	0.0378
Recall lớp 1	0.8313	0.7439	0.9103	0.0420
Precision lớp 1	0.8734	0.7949	0.9419	0.0377
Recall lớp 0	0.7059	0.5455	0.8485	0.0780
Precision lớp 0	0.6316	0.4857	0.7858	0.0784
Macro F1	0.7593	0.6762	0.8403	0.0430

Phân phối bootstrap của recall lớp có bệnh



Hình 7: Phân phối bootstrap của recall lớp có bệnh



Hình 8: Khoảng tin cậy bootstrap 95% cho sáu chỉ số trên tập kiểm tra

Recall của lớp có bệnh đạt 0,831, với khoảng tin cậy Bootstrap 95% từ 0,744 đến 0,910. Đối với Accuracy và Macro F1, các khoảng tin cậy tương ứng lần lượt là [0,718; 0,872] và [0,676; 0,840]. Đáng chú ý, cận dưới của khoảng tin cậy đối với Recall lớp có bệnh vẫn đạt khoảng 0,744, cho thấy khả năng phát hiện các trường hợp mắc bệnh của mô hình được duy trì ở mức tương đối cao ngay cả khi xét đến biến thiên do thành phần của tập kiểm tra.

Độ bất định lớn nhất vẫn nằm ở lớp không bệnh. Recall lớp 0 có khoảng tin cậy [0,545; 0,849], còn precision lớp 0 có khoảng [0,486; 0,786]. Nguyên nhân chính là tập kiểm tra chỉ có 34 người không bệnh, nên một vài dự đoán thay đổi cũng làm chỉ số dao động đáng kể.

9 Kết luận

Nghiên cứu này đã xây dựng và đánh giá một quy trình phân loại nguy cơ mắc bệnh gan trên bộ dữ liệu ILPD bằng hồi quy logistic. Bên cạnh việc lựa chọn mô hình, nghiên cứu thiết lập một quy trình đánh giá nhất quán: kiểm định chéo phân tầng năm phần được dùng để lựa chọn cấu hình và tạo dự đoán *out-of-fold* phục vụ lựa chọn ngưỡng, còn bootstrap được áp dụng trên tập kiểm tra độc lập để ước lượng khoảng tin cậy của các chỉ số đánh giá.

Trên tập kiểm tra độc lập gồm 117 quan sát, mô hình đạt Accuracy 0,795,

recall lớp có bệnh 0,831, precision lớp có bệnh 0,873 và Macro F1 0,759. Mô hình phát hiện đúng 69/83 ca bệnh, bỏ sót 14 ca và cảnh báo nhầm 10/34 người không bệnh. So với ngưỡng mặc định 0,5, recall lớp có bệnh tăng từ 0,747 lên 0,831 và Macro F1 tăng từ 0,748 lên 0,759, trong khi recall lớp không bệnh giảm từ 0,824 xuống 0,706. Như vậy, điều chỉnh ngưỡng đã giảm số ca bệnh bị bỏ sót mà không làm suy giảm chất lượng phân loại tổng hợp; đánh đổi chủ yếu là số cảnh báo nhầm tăng lên.

Phân tích bootstrap với 2.000 mẫu ước lượng khoảng tin cậy 95% của recall lớp có bệnh là $[0,744; 0,910]$; các khoảng tương ứng của Accuracy và Macro F1 lần lượt là $[0,718; 0,872]$ và $[0,676; 0,840]$. Kết quả này cho thấy khả năng phát hiện ca bệnh được duy trì ở mức tương đối tốt trên dữ liệu khảo sát. Tuy nhiên, các chỉ số của lớp không bệnh còn có khoảng tin cậy rộng do tập kiểm tra chỉ gồm 34 quan sát ở lớp này. Vì vậy, ước lượng điểm cần được diễn giải cùng khoảng tin cậy và không nên suy rộng trực tiếp cho một quần thể lớn hơn.

Nhìn chung, hồi quy logistic kết hợp SMOTE cho thấy tiềm năng hỗ trợ sàng lọc trên bộ dữ liệu ILPD, đặc biệt khi ngưỡng phân loại được xác định theo ưu tiên giảm bỏ sót ca bệnh.

Hạn chế. Bộ dữ liệu chỉ có 583 quan sát và tập kiểm tra có 34 trường hợp không bệnh, khiến khoảng tin cậy của các chỉ số lớp 0 còn rộng. Bootstrap trong báo cáo giữ cố định mô hình nên chưa phản ánh biến thiên do huấn luyện lại. Ngoài ra, đây là dữ liệu quan sát từ một khu vực cụ thể; kết quả hiện tại chỉ có ý nghĩa đánh giá mô hình trên bộ ILPD, chưa đủ để suy rộng thành hiệu quả chẩn đoán lâm sàng.

Hướng phát triển. Các nghiên cứu tiếp theo có thể lặp lại quá trình huấn luyện trên nhiều cách chia dữ liệu, huấn luyện lại mô hình trong từng vòng bootstrap và kiểm định trên một tập dữ liệu ngoài. Việc so sánh với các mô hình phi tuyến cũng nên được thực hiện với cùng cách chia dữ liệu và cùng tiêu chí đánh giá để kết quả so sánh có ý nghĩa.

Tài liệu tham khảo

1. Benjamini, Y., & Hochberg, Y. (1995). Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing. *Journal of the Royal Statistical Society, Series B*, 57(1), 289–300.
2. Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321–357.
3. Efron, B., & Tibshirani, R. J. (2000). An introduction to the bootstrap (pp. p-436). Boca Raton, Florida.
4. Kohavi, R. (1995, August). A study of cross-validation and bootstrap for accuracy estimation and model selection. In *Ijcai* (Vol. 14, No. 2, pp. 1137-1145).
5. Hoerl, A. E., & Kennard, R. W. (1970). Ridge Regression: Biased Estimation for Nonorthogonal Problems. *Technometrics*, 12(1), 55–67.
6. Ramana, B., & Venkateswarlu, N. (2022). ILPD (Indian Liver Patient Dataset) [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.24432/C5D02C>